App Dev Project Report

1. Student Details

Name: Shaurya Shukla
Roll Number: 24F2005423
Email: 24f2005423@ds.study.iitm.ac.in

About Me:
I am a student interested in web application development and backend systems. I enjoy designing applications that solve real-world problems using modern web technologies. This project focuses on building a complete **Placement Portal system** that connects students, companies, and administrators through a structured web platform.

---

2. Project Details

Project Title:
Placement Portal System

Problem Statement:
To design and develop a web-based application that manages the placement process for students and companies. The system allows students to apply for placement drives, companies to create job postings, and administrators to manage approvals and monitor the entire placement workflow.

Approach:

The application was developed using **Flask** as the backend web framework. The system follows a structured architecture with separate modules for database handling, templates, and static resources.

The platform supports three types of users:

- **Admin**
- **Companies**
- **Students**

The workflow is:

1. Companies register and wait for admin approval.
2. Approved companies create placement drives.
3. Students register, build profiles, and apply for drives.

4. Admin manages students, companies, and applications.

The application uses **SQLite** as the database and provides a simple and responsive user interface using HTML templates.

---

3. AI / LLM Declaration

I used **ChatGPT** for limited assistance such as:

- Improving code readability
- Understanding Flask routing and database connections
- Basic CSS/Frontend coding idea
- Minor debugging suggestions

The extent of AI usage is approximately **10–15%** and was limited to learning assistance and documentation formatting.

All major design decisions, coding, database structure, and integration were implemented manually.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLite	Lightweight relational database
SQLite3 (Python module)	Database connection and queries
Jinja2	Template engine for dynamic HTML pages
HTML / CSS	Frontend page structure and styling
JavaScript	Client-side interactivity
Werkzeug	Password hashing and security
Bootstrap	Responsive UI components

---

5. Database Schema / ER Diagram

Tables:

Admin — stores administrator details
(id, name, email, password)

Students — stores student profile details
(id, name, email, password, roll_no, branch, cgpa, phone, skills, resume, created_at)

Companies — stores company registration information
(id, company_name, email, password, hr_contact, website, industry, description, approval_status, created_at)

Placement Drive — stores job drives created by companies
(id, company_id, job_title, job_description, eligibility_criteria, package, location, application_deadline, created_at)

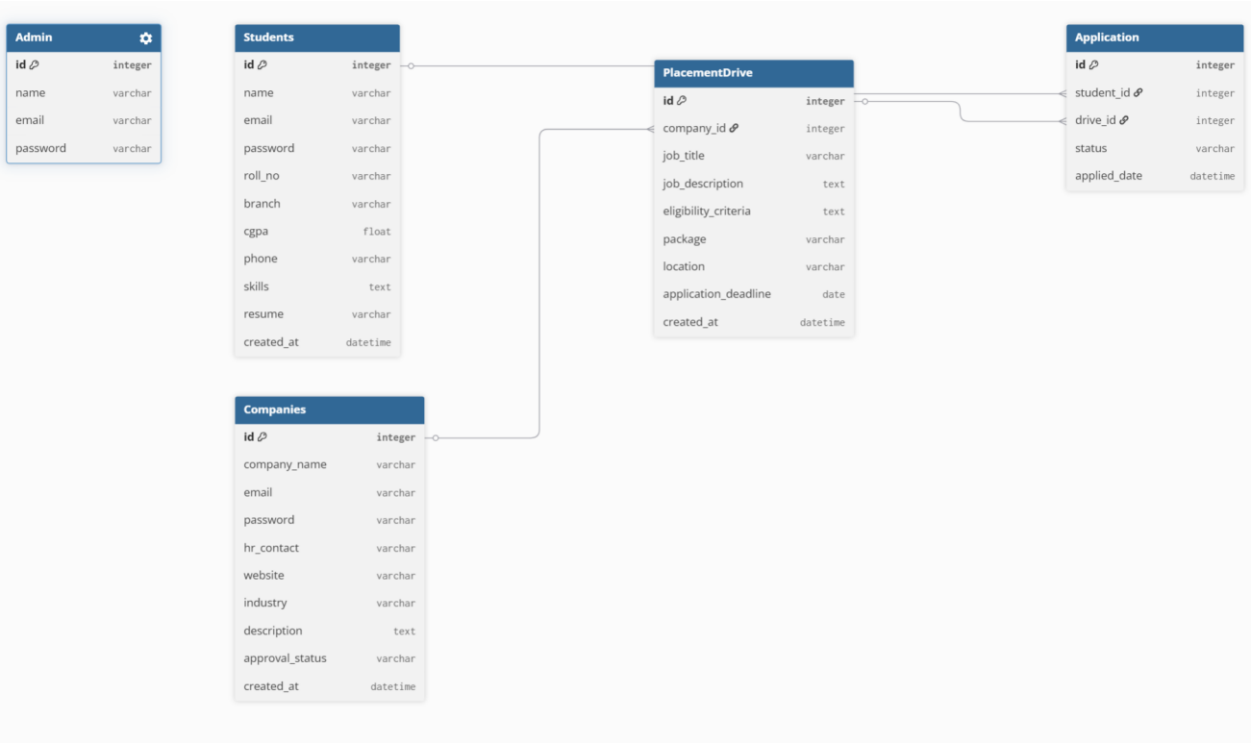
Application — stores student applications for placement drives
(id, student_id, drive_id, status, applied_date)

Relationships:

One-to-Many → Company → Placement Drive
One company can create multiple placement drives.

One-to-Many → Student → Application
One student can apply to multiple placement drives.

One-to-Many → Placement Drive → Application
One placement drive can receive applications from multiple students.



6. API / System Endpoints

Endpoint	Method	Description
/login	POST	Authenticate user login
/register_student	POST	Register a new student
/register_company	POST	Register a company
/student/dashboard	GET	Student dashboard with available drives
/company/dashboard	GET	Company dashboard
/admin/dashboard	GET	Admin control panel
/create_drive	POST	Company creates placement drive
/apply	POST	Student applies for job drive

---

7. Architecture and Features

Architecture Overview

app.py – main Flask application entry point and route handling

models.py – database models defining tables such as students, companies, drives, and applications

config.py – configuration settings such as database path, secret keys, and application settings

utils.py – helper functions for tasks like validation, file handling, and common utilities

database.py – database initialization and connection setup for SQLite

/templates – Jinja2 HTML templates used to render dynamic pages

/static – CSS, JavaScript, and uploaded files such as resumes

README.md – project documentation containing setup instructions and project overview

Implemented Features

- **User registration and login**
- **Student profile creation and resume upload**
- **Company registration and job drive creation**

- **Students applying to placement drives**
- **Application tracking system**
- **Admin dashboard for managing companies, students, and drives**
- **Responsive user interface using HTML and Bootstrap**
- **Secure password hashing and session handling**

Additional Features

- **Resume upload and storage**
- **Company approval system by admin**
- **Application status tracking**
- **Student blacklist functionality**
- **Prevents duplicate applications to the same drive**

---

8. Video Presentation

Drive Link:
(video_link)